

Universidad Rafael Landívar.
Licenciatura en Ingeniería en Informática y Sistemas.
Manejo e Implementación de Archivos, Sección 02.
Docente: Ing. Luis Rojas

ACTIVIDAD NO. 1

SEMANA #13

APIS CONCEPTOS

Estudiante: López Castillo, Kenneth Isaías Yahid
Carné: 1138025

Guatemala de la Asunción 1 de octubre del 2026

Actividad No. 1 Semana #13 - APIs

A) Tipos de API

- API Pública (Open API)
 - ¿Qué es?: es una API abierta para cualquier persona o desarrollador. Normalmente se debe registrar uno, dan una llave (API Key) y ya se puede usar. Unas son gratis y otras cobran según el uso.
 - Para qué sirve?: para que otros programadores usen los datos o servicios de una empresa en sus propias apps.
 - Ejemplo real: Google Maps API. Permite que una app de delivery muestre mapas y rutas sin tener que crear su propio mapa. También la API de OpenWeather.
- API Interna (Private API)
 - ¿Qué es?: es una API que solo usa la misma empresa, entre sus propios sistemas o equipos. Desde afuera nadie puede entrar.
 - Para qué sirve?: para que las partes de un sistema se hablen entre sí, sin exponer nada al público.
 - Ejemplo real: la API que usa la app de un banco para consultar el saldo de sus clientes en sus servidores. Solo la app oficial la usa.

• API de Socios (Partner API)

→ Qué es?: es una API que se comparte solo con empresas aliadas, con contrato o permiso. No es pública pero tampoco es solo interna.

→ Para qué sirve?: para que dos empresas trabajen juntas de forma controlada y segura.

→ Ejemplo real: Una tienda en línea que se conecta con Stripe o PayPal para cobrar. También un banco que da acceso a una empresa aliada para procesar pagos o consultar movimientos.

• API Compuesta (Composite API)

→ Qué es?: es una API que junta varias llamadas en una sola. En vez de hacer 3 o 4 peticiones por separado, se hace una y ella se encarga del resto.

→ Para qué sirve?: para ahorrar tiempo y hacer la app más rápida, porque se hacen menos viajes al servidor.

→ Ejemplo real: una app de viajes que con una sola petición trae vuelos, hotel y clima, por detrás está llamando a 3 APIs distintas. Otro ejemplo es Salesforce, que permite hacer varias operaciones en una sola llamada.

B) Estilos arquitectónicos

• REST

→ ¿Cómo funciona?: cada cosa (usuarios, productos, etc.) tiene su dirección (URL), y con los verbos se le dice qué hacer:

* GET = para pedir

* POST = para crear

* PUT = para actualizar

* DELETE = para borrar

Responde en JSON y cada petición es independiente.

• GraphQL

→ ¿Cómo funciona?: se escribe una consulta diciendo exactamente qué datos se quiere, y el servidor devuelve todo eso. Todo se hace por una sola dirección (endpoint).

→ ¿Dónde se usa más?: apps móviles y redes sociales, donde se necesitan datos específicos de varias partes.

→ Ventaja: No le llega información de más al usuario ni se deben hacer muchas peticiones.

• SOAP

→ ¿Cómo funciona?: manda mensajes en formato XML, con reglas muy estrictas y un formato fijo. Es más pesado y formal que REST, pero muy ordenado y seguro.

→ ¿Dónde se usa más?: en bancos, aseguradoras, gobierno y sistemas viejos, donde la seguridad y las reglas importan.

→ Ventaja: tiene estándares fuertes de seguridad y manejo de errores.

- gRPC

- Cómo funciona?: un servicio llama a funciones de otro como si fueran locales. los datos viajan en formato binario, así que es más rápido y liviano. lo creó Google.

- Dónde se usa más?: comunicación entre microservicios dentro de una empresa y sistemas que necesitan mucha velocidad.

- Ventaja: es muy rápido y eficiente.

- WebSocket

- Cómo funciona?: abre una conexión que se queda abierta, y el cliente y servidor se mandan datos en tiempo real, sin tener que volver a preguntar cada vez.

- Dónde se usa más?: chats, juegos en línea, notificaciones en vivo.

- Webhooks

- Cómo funciona?: en vez de estar preguntando si "ya pasó?", el sistema le avisa al usuario cuando algo ocurre, mandando una petición a una URL que el ^{servidor} ~~usuario~~ le dio.

- Dónde se usa más?: avisos de pagos, Github avisando de un nuevo push.

- Diferencias: WebSocket es una conversación abierta y constante. Webhook es un aviso que llega solo cuando pasa algo.

c) Conceptos relacionados

- Payload
- Qué es?: son los datos importantes que se mandan dentro de una petición o respuesta. Es el "contenido" del mensaje, sin contar lo demás como el encabezado.
- Ejemplo: { "nombre": "Kenneth", "carrera": "Sistemas" }

• Request (petición)

- Qué es?: es el mensaje que el cliente le manda al servidor para su solicitud. Lleva el método, la URL, encabezados y a veces un payload.

- Ejemplo: GET <https://api.ejemplo.com/usuarios/1>
(le pide el usuario número 1)

• Response (respuesta)

- Qué es?: es lo que el servidor contesta. Tiene un código de estado, encabezados y normalmente datos.

- Ejemplo: código 200 OK con { "id": 1, "nombre": "Kenneth" }

- * 200 = todo bien
- * 201 = creado
- * 400 = petición mala
- * 401 = sin autorización
- * 404 ~~no~~ = no existe
- * 500 = error del servidor

- cURL
- Qué es? es una herramienta de la terminal para hacer peticiones a una API escribiendo en comando. Sirve para probar APIs sin necesidad de programar una app.

→ Ejemplo: * para pedir datos = `curl https://api.ejemplo.com/usuarios/1`

* para enviar datos = `curl -x POST https://api.ejemplo.com/usuarios -H "Content-Type: application/json" -d '{"nombre": "Kenneth"}'`

Tipos de Autorización en Postman

- No Auth: sin autorización, se usa APIs públicas que no piden nada.
- API Key: una llave que se manda en el header o en la URL, para que el servidor sepa quién somos.
- Bearer Token: se manda un token (como en JWT) en el header y el servidor lo valida. Muy usado en APIs modernas.
- Basic Auth: usuario y contraseña mandados en cada petición. Es simple, pero menos seguro si no se usa HTTPS.
- OAuth 2.0: se entra con otra cuenta y nos dan un token con permisos limitados, sin compartir la contraseña.
- Digest Auth: Parecido a Basic Auth, pero la contraseña no viaja en texto plano.